1. State Machines and Event-Driven Systems

Why it matters. Any system whose response depends on current state benefits from explicit states, events, transitions, guards, and terminal conditions. State machines turn control flow hidden in loops and conditionals into inspectable data and rules.

Target capability. You can identify state-machine-shaped problems, draw transition diagrams, separate events from commands, model progress explicitly, and test legal and illegal transitions.

Traffic Light Controller

Goal	Model a four-phase traffic light that advances only through legal transitions.
Learn	Enum states, transition table, terminal-independent loop, invariants.
Constraints	Console only. No framework. An invalid transition must fail clearly.
Done when	A transition diagram matches the implementation; every state has a tested successor; 100 cycles preserve the sequence.
Budget	2-3 hours
Stretch	Add a pedestrian request that affects a future transition rather than changing the light immediately.

Vending Machine

Goal	Accept selections and coins, dispense only when inventory and funds are sufficient, and return change or cancel.
Learn	State plus accumulated data, guards, commands versus events, error states.
Constraints	Use integer cents. No GUI or database.
Done when	Tests cover exact payment, excess payment, insufficient funds, unavailable item, cancellation, and empty inventory.
Budget	4-6 hours
Stretch	Add a maintenance mode that cannot be entered while a transaction is active.

Elevator Dispatcher

Goal	Simulate one elevator serving floor requests with doors, direction, and movement states.
Learn	Hierarchical state, event queue, transition priority, safety invariants.
Constraints	One elevator, 10 floors, deterministic simulation.
Done when	The elevator never moves with doors open, never leaves valid floors, and eventually services a finite request set.
Budget	5-8 hours
Stretch	Add cabin and hall calls with different scheduling rules.

Resumable Card Dealer

Goal	Deal one card per Advance command to four players and stop after two dealing passes.
Learn	Persisted loop progress, automatic transitions, observable intermediate state.
Constraints	No poker betting, evaluator, GUI framework, or animation.
Done when	After each command exactly one card changes ownership; eight valid advances complete dealing; later advances are rejected.
Budget	3-5 hours
Stretch	Add explicit blind-posting phases before dealing.

2. Object and Domain Modeling

Why it matters. Software models are useful when they preserve the vocabulary, identity, invariants, and lifecycle of the problem rather than simply mirroring screen widgets or database rows.

Target capability. You can distinguish entities from values, place invariants near the data they protect, model lifecycle explicitly, and choose aggregate boundaries without premature patterns.

Library Lending Model

Goal	Model books, physical copies, members, loans, reservations, and return deadlines.
Learn	Entity identity, value objects, lifecycle, multiplicity, invariants.
Constraints	In-memory model only. No user interface.
Done when	A copy cannot have two active loans; a returned loan remains historically inspectable; reservations follow a documented order.
Budget	4-7 hours
Stretch	Introduce lost copies and replacement charges.

Money and Ledger

Goal	Implement immutable money values and a double-entry ledger for transfers between accounts.
Learn	Value semantics, precision, invariants, audit trail, balancing.
Constraints	No floating point; use integer minor units and explicit currencies.
Done when	Every transaction balances debits and credits; cross-currency addition is rejected; history is immutable.
Budget	5-8 hours
Stretch	Add reversal transactions rather than deleting mistakes.

Warehouse Inventory

Goal	Track products, stock locations, receipts, reservations, picks, and shipments.
Learn	Aggregate boundaries, temporal state, stock versus available stock.
Constraints	No database. All mutations occur through named domain operations.
Done when	Available quantity never becomes negative; reserved stock cannot be shipped twice; a shipment has a legal lifecycle.
Budget	6-10 hours
Stretch	Model transfers between locations as two linked movements.

Meeting-Room Scheduler

Goal	Schedule rooms and attendees while rejecting overlapping reservations and invalid time ranges.
Learn	Intervals, identities, domain services, conflict rules.
Constraints	Use timezone-aware timestamps or clearly document a single fixed timezone.
Done when	Boundary-touching meetings behave according to a documented rule; overlap tests cover containment and partial overlap.
Budget	4-7 hours
Stretch	Add recurring meetings and exception dates.

3. Data Structures and Algorithms

Why it matters. Data structure choice determines which operations are easy, expensive, or impossible. Algorithmic thinking provides language for predicting cost before optimizing blindly.

Target capability. You can select structures based on required operations, reason with Big-O at a practical level, implement and test traversals, and measure rather than assume.

Autocomplete Index

Goal	Build prefix-based suggestions over a dictionary with ranking by frequency.
Learn	Trie or sorted index, prefix search, complexity, memory trade-offs.
Constraints	Implement two approaches and compare them on at least 50,000 generated words.
Done when	Benchmarks report build time, query time, and approximate memory; README explains the chosen approach.
Budget	6-10 hours
Stretch	Support insertion and deletion after construction.

Maze Solver

Goal	Load an ASCII maze and find a path from start to finish.
Learn	Graph representation, BFS, DFS, parent maps, shortest paths.
Constraints	Implement both BFS and DFS without a graph library.
Done when	BFS returns a shortest unweighted path; unreachable mazes are reported; paths are rendered back into ASCII.
Budget	4-7 hours
Stretch	Add weighted terrain and Dijkstra's algorithm.

Least-Recently-Used Cache

Goal	Implement a fixed-capacity cache with O(1)-average lookup, insertion, and eviction.
Learn	Hash map plus linked order, invariants, complexity contracts.
Constraints	Do not use functools.lru_cache or OrderedDict for the core implementation.
Done when	Tests cover update, access promotion, eviction, capacity one, and duplicate keys; invariant checks run after each operation.
Budget	4-6 hours
Stretch	Add hit/miss metrics and resize behavior.

Dependency Planner

Goal	Order tasks according to dependencies and explain cycles.
Learn	Directed graphs, topological sorting, cycle detection, diagnostics.
Constraints	Input is a simple text or JSON task list.
Done when	Produces a valid order for DAGs; returns a concrete cycle path for cyclic input; tests verify ordering constraints.
Budget	5-8 hours
Stretch	Compute parallelizable layers.

4. Parsing and Language Processing

Why it matters. Parsers convert untrusted text into structured meaning. The same skills appear in configuration, protocols, compilers, search queries, templates, and command interfaces.

Target capability. You can separate lexing, parsing, semantic validation, and evaluation; design a grammar; build an AST; and produce useful source-positioned errors.

Arithmetic Expression Evaluator

Goal	Parse and evaluate expressions with precedence, parentheses, unary minus, and variables.
Learn	Tokens, recursive descent, precedence, AST, evaluation.
Constraints	Do not use eval(). Write a grammar before implementation.
Done when	Correctly distinguishes 2+3*4 from (2+3)*4 and reports line/column for malformed input.
Budget	6-10 hours
Stretch	Add functions such as min() and max().

Configuration Language

Goal	Parse a small configuration format with sections, scalars, lists, comments, and duplicate-key errors.
Learn	Concrete syntax, semantic validation, diagnostics, round-tripping.
Constraints	Do not reuse JSON/TOML/YAML parsers.
Done when	Parser returns typed values; duplicate keys and invalid literals include precise locations; sample config round-trips.
Budget	6-10 hours
Stretch	Add includes with cycle detection.

Search Query Parser

Goal	Parse queries such as status:open owner:max "exact phrase" -label:wip into an AST.
Learn	Token ambiguity, quoting, unary operators, domain-specific language.
Constraints	Parsing only; no database required.
Done when	AST distinguishes free text, filters, negation, and phrases; malformed quotes produce actionable errors.
Budget	4-7 hours
Stretch	Compile the AST to a predicate over in-memory records.

Mini Template Engine

Goal	Render text with variables, escaping, if-blocks, and simple loops.
Learn	Scanning, nested structures, AST execution, escaping boundaries.
Constraints	No Jinja2 or equivalent engine.
Done when	Nested blocks work; missing variables follow a documented policy; HTML escaping is tested.
Budget	7-12 hours
Stretch	Add source maps from output segments to template locations.

5. Persistence and Data Management

Why it matters. Applications must preserve meaning across process lifetimes. Persistence introduces schema, identity, transactions, migration, recovery, and consistency problems that do not exist in memory.

Target capability. You can choose a representation, design normalized relational data, use transactions, migrate schemas safely, and distinguish authoritative data from caches.

Versioned Save File

Goal	Persist a small application state to JSON and load older schema versions.
Learn	Serialization, validation, schema versions, migrations, atomic writes.
Constraints	Write to a temporary file and atomically replace the target.
Done when	Corrupt files fail without destroying the previous save; versions 1 and 2 both load into the current model.
Budget	4-7 hours
Stretch	Add checksums and backup rotation.

SQLite Expense Ledger

Goal	Store accounts, categories, transactions, and split line items in SQLite.
Learn	Relational modeling, primary/foreign keys, normalization, queries.
Constraints	Use sqlite3 directly before trying an ORM.
Done when	Foreign keys are enabled; monthly/category reports are query-driven; invalid references are rejected.
Budget	7-12 hours
Stretch	Add recurring transaction generation.

Transactional Seat Reservation

Goal	Reserve a limited seat exactly once under competing attempts.
Learn	Transactions, uniqueness, race conditions, isolation expectations.
Constraints	Use SQLite and two concurrent connections or processes.
Done when	Competing reservations cannot both succeed; failures leave consistent data; behavior is explained.
Budget	6-10 hours
Stretch	Add temporary holds with expiration.

Schema Migration Runner

Goal	Apply numbered SQL migrations and record applied versions.
Learn	Forward migration, idempotence boundaries, checksums, failure recovery.
Constraints	No migration framework.
Done when	Migrations run in order, never silently change after application, and stop safely on failure.
Budget	5-8 hours
Stretch	Add a dry-run and status command.

6. Testing and Debugging

Why it matters. Testing is executable reasoning about behavior; debugging is disciplined hypothesis reduction. Both protect change and expose mistaken models rather than merely increasing coverage numbers.

Target capability. You can select test levels, isolate nondeterminism, use property-based thinking, construct useful fakes, and reduce defects to minimal reproducible cases.

Characterization Test Rescue

Goal	Take a deliberately messy legacy-style module and capture existing behavior before refactoring it.
Learn	Characterization tests, seams, regression safety, behavior versus implementation.
Constraints	Do not change production behavior before the first test suite passes.
Done when	Refactor reduces complexity while the characterization suite remains green; undocumented quirks are listed.
Budget	5-8 hours
Stretch	Replace one characterization test with a clearer specification test.

Property-Based Roman Numerals

Goal	Implement integer/Roman conversion and test general properties, not only examples.
Learn	Round-trip properties, generators, shrinking mindset, invariants.
Constraints	Use Hypothesis if desired, but first write the properties in plain language.
Done when	All values 1-3999 round-trip; canonical output is unique; invalid forms are rejected.
Budget	3-6 hours
Stretch	Write a custom input generator.

Deterministic Time and Randomness

Goal	Refactor a token/expiry service so tests control time and random generation.
Learn	Dependency injection, clocks, random sources, deterministic testing.
Constraints	No sleeps in tests and no monkey-patching global time.
Done when	Expiry-boundary tests are exact and reliable; random tokens are reproducible under a fake source.
Budget	4-6 hours
Stretch	Add a simulation test covering thousands of expirations.

Debugging Casebook

Goal	Create and diagnose five defects: off-by-one, stale state, aliasing, race, and resource leak.
Learn	Reproduction, logging, breakpoints, hypothesis tables, fault localization.
Constraints	For each defect, record hypotheses before the fix.
Done when	Each case includes a minimal reproduction, root cause, rejected hypotheses, fix, and regression test.
Budget	8-12 hours
Stretch	Give one case to another developer without the answer.

7. Architecture and Modular Design

Why it matters. Architecture is the set of boundaries and decisions that make some changes easy and others expensive. Small exercises make dependency direction and integration seams concrete without requiring enterprise-scale systems.

Target capability. You can identify stable versus volatile concerns, define narrow interfaces, keep domain rules independent of delivery mechanisms, and justify boundaries through change scenarios.

Notification Ports

Goal	Build a reminder service that can send through console, email fake, or file output without changing reminder rules.
Learn	Ports and adapters, dependency inversion, test doubles.
Constraints	No dependency-injection framework.
Done when	Domain tests run with an in-memory port; adding a new delivery adapter changes no domain rules.
Budget	4-7 hours
Stretch	Add retries in the adapter without leaking policy into the domain.

Two Front Ends, One Core

Goal	Expose the same task-list domain through a CLI and a minimal HTTP API.
Learn	Application services, delivery adapters, shared core, DTO boundaries.
Constraints	Neither front end may import the other.
Done when	The same acceptance tests execute through both front ends; domain code has no argparse or HTTP imports.
Budget	8-12 hours
Stretch	Add a third scripted batch adapter.

Plugin-Based Exporter

Goal	Export a report to text, CSV, and JSON through discoverable plugins.
Learn	Stable extension point, interface contracts, plugin loading, failure isolation.
Constraints	Core report generation must not branch on output format.
Done when	A new exporter can be added without editing core dispatch logic; plugin failures identify the provider.
Budget	6-10 hours
Stretch	Load external plugins from a configured directory.

Architecture Change Drill

Goal	Implement a small application, then apply three planned change requests and measure touched modules.
Learn	Coupling, cohesion, change amplification, architecture decision records.
Constraints	Write predictions before each change.
Done when	A short report compares predicted and actual change impact and names one boundary to keep or remove.
Budget	8-12 hours
Stretch	Repeat after a deliberate architectural refactor.

8. Concurrency and Asynchronous Programming

Why it matters. Concurrent programs must remain correct under many possible interleavings. Async programs also require cancellation, timeouts, ownership, and pressure control rather than merely “running things at once.”

Target capability. You can choose threads, processes, or async tasks; protect shared state; use queues; propagate cancellation; and reproduce concurrency failures.

Producer-Consumer Pipeline

Goal	Process jobs using bounded queues and multiple workers.
Learn	Queue ownership, backpressure, worker lifecycle, sentinel shutdown.
Constraints	Use the standard library only.
Done when	No jobs are lost or processed twice; bounded capacity slows producers; shutdown completes without hanging.
Budget	5-8 hours
Stretch	Add per-stage metrics and retries.

Concurrent Bank Transfer

Goal	Expose and then eliminate a race in transfers between accounts.
Learn	Critical sections, lock ordering, invariants, deadlock prevention.
Constraints	First preserve a failing stress test that demonstrates the race.
Done when	Total money remains constant across repeated concurrent runs; lock ordering is documented.
Budget	5-8 hours
Stretch	Compare coarse- and fine-grained locking performance.

Async Web Fetch Simulator

Goal	Run many simulated I/O tasks with concurrency limits, deadlines, and cancellation.
Learn	asyncio tasks, semaphores, timeout, structured cleanup.
Constraints	Simulation may use sleep; no external service required.
Done when	Concurrency never exceeds the configured limit; timeout and cancellation leave no orphan tasks.
Budget	5-8 hours
Stretch	Add retry with exponential backoff and jitter.

Debounced File Indexer

Goal	Consume rapid file-change events and rebuild an index only after activity settles.
Learn	Debouncing, event coalescing, cancellation, eventual consistency.
Constraints	A simulated event source is sufficient.
Done when	Bursts trigger one rebuild; changes during rebuild cause a later rebuild; shutdown flush policy is documented.
Budget	6-10 hours
Stretch	Use a real filesystem watcher afterward.

9. Networking and Distributed Systems

Why it matters. Networked software operates across unreliable boundaries. Messages may be delayed, duplicated, truncated, reordered, or lost; remote failure cannot be treated like a normal function call.

Target capability. You can define framing and protocols, reason about HTTP semantics, make operations idempotent, apply timeouts and retries, and design for partial failure.

Length-Prefixed Chat Protocol

Goal	Build a TCP chat server and clients using explicit message framing.
Learn	Sockets, streams versus messages, framing, disconnect handling.
Constraints	Do not assume one recv() equals one message.
Done when	Handles split and combined packets, multiple clients, clean disconnects, and malformed lengths.
Budget	8-12 hours
Stretch	Add nicknames and server-side commands.

HTTP Task Service

Goal	Implement a small REST-style task service with correct status codes and validation.
Learn	HTTP methods, resources, representations, validation, error bodies.
Constraints	Use a minimal framework, but write the HTTP contract first.
Done when	Create/read/update/delete behavior is covered by black-box tests; malformed input never causes 500.
Budget	8-12 hours
Stretch	Add conditional updates with ETags/version numbers.

Idempotent Payment Simulator

Goal	Accept payment requests with idempotency keys so safe retries do not charge twice.
Learn	At-least-once delivery, deduplication, idempotency, durable result replay.
Constraints	Persist accepted keys and outcomes.
Done when	Repeated identical requests return the original result; conflicting reuse of a key is rejected.
Budget	6-10 hours
Stretch	Add a simulated timeout after commit but before response.

Replicated Counter Simulation

Goal	Simulate two nodes accepting offline increments and later merging.
Learn	Eventual consistency, conflict-free merge, logical clocks or CRDT intuition.
Constraints	Simulation only; no real network needed.
Done when	Merge is associative, commutative, and idempotent; nodes converge after exchanging state.
Budget	5-8 hours
Stretch	Implement a per-node grow-only counter and a positive/negative counter.

10. Operating Systems and Systems Programming

Why it matters. Programs live inside operating systems and consume finite resources. Process boundaries, file descriptors, streams, signals, encodings, and binary formats explain many “mysterious” production failures.

Target capability. You can work with processes and pipes, manage resources explicitly, inspect binary data, handle signals, and distinguish filesystem paths from file contents.

Pipeline Shell

Goal	Execute a two- or three-command pipeline and connect standard streams.
Learn	Processes, stdin/stdout/stderr, pipes, exit status.
Constraints	Use subprocess APIs; no shell=True for the core path.
Done when	Captures exit codes, propagates errors, avoids deadlock on large output, and closes resources.
Budget	5-8 hours
Stretch	Add redirection to and from files.

Binary File Inspector

Goal	Parse a documented binary format header and display fields safely.
Learn	Bytes, endianness, struct layout, offsets, validation.
Constraints	Choose a modest format such as BMP header or a custom format.
Done when	Rejects truncated and invalid files without uncontrolled exceptions; sample bytes are explained.
Budget	5-8 hours
Stretch	Write a file and read it back.

Atomic Log Rotator

Goal	Append logs, rotate by size, and preserve a bounded set of archives.
Learn	File handles, rename semantics, atomicity, cleanup, failure recovery.
Constraints	Do not lose existing logs when rotation fails.
Done when	Tests simulate threshold crossings and partial failures; archive naming and retention are deterministic.
Budget	5-8 hours
Stretch	Coordinate two writer processes.

Graceful Process Shutdown

Goal	Run a worker process that handles termination, stops accepting work, and completes or cancels in-flight tasks.
Learn	Signals, lifecycle, cleanup, exit codes.
Constraints	Test by launching and terminating a real subprocess.
Done when	No corrupted output or orphan process remains; forced-shutdown deadline is documented.
Budget	5-8 hours
Stretch	Add parent-child heartbeat monitoring.

11. Security Engineering

Why it matters. Security is a property of design, implementation, configuration, dependencies, and operation. It begins with assets and trust boundaries, not with adding cryptography at the end.

Target capability. You can threat-model a small system, enforce authorization server-side, validate hostile input, handle secrets and passwords safely, and recognize common web risks.

Threat Model a Notes Service

Goal	Draw trust boundaries, identify assets and abuse cases, and prioritize mitigations before implementation.
Learn	Threat modeling, attack surface, likelihood/impact, trust boundaries.
Constraints	Use a simple data-flow diagram and a method such as STRIDE.
Done when	At least 10 threats are ranked; each high-priority threat has a preventive or detective control and verification idea.
Budget	3-5 hours
Stretch	Revisit the model after implementing the service.

Authorization Matrix

Goal	Implement owner, editor, and viewer permissions for shared documents.
Learn	Authentication versus authorization, deny by default, object-level access control.
Constraints	All checks occur in the service layer, not only the UI.
Done when	A table of subjects/actions/resources maps to tests; horizontal privilege escalation attempts fail.
Budget	5-8 hours
Stretch	Add revocation and inherited organization roles.

Secure Password Store

Goal	Register and verify passwords using a standard password-hashing API and migration-friendly parameters.
Learn	Salt, work factor, verification, secret handling, timing concerns.
Constraints	Never implement your own cryptography; use a vetted library such as Argon2id/bcrypt support.
Done when	Plaintext passwords are never stored or logged; verification and rehash-on-policy-change are tested.
Budget	4-7 hours
Stretch	Add password reset tokens with expiration and one-time use.

Injection and Misconfiguration Lab

Goal	Build a deliberately vulnerable local app, write exploits for SQL injection and unsafe configuration, then repair and regress-test it.
Learn	Parameterized queries, secure defaults, error handling, logging.
Constraints	Local isolated environment only; no real targets or credentials.
Done when	Each exploit has a failing security test before remediation; repaired code prevents the exploit without hiding errors.
Budget	6-10 hours
Stretch	Add a dependency and supply-chain inventory.

12. User Interfaces, Graphics, and Human-Computer Interaction

Why it matters. Interactive software combines input, application state, layout, rendering, feedback, accessibility, and latency. Correctness includes what the user can understand and recover from.

Target capability. You can separate state from rendering, reason about coordinates and font metrics, build input modes, design feedback, and test interaction without overcoupling domain logic.

Character Screen Toolkit

Goal	Build a fixed-cell Pygame text screen with clipping, colors, horizontal/vertical writing, and glyph diagnostics.
Learn	Coordinate systems, font metrics, rasterization, drawing primitives.
Constraints	Screen owns cells; poker/domain code is absent.
Done when	A diagnostic page shows supported glyphs, baselines, clipping, and box behavior at configured sizes.
Budget	6-10 hours
Stretch	Add a bitmap-font backend.

Keyboard-Only Form

Goal	Build a small form with focus order, editing, validation, submit, and cancel.
Learn	Focus state, input modes, feedback, accessibility, event translation.
Constraints	Operable without a mouse; validation errors preserve entered data.
Done when	Tab order is predictable; invalid fields are identified; Escape and Enter follow documented rules.
Budget	6-10 hours
Stretch	Add configurable key bindings.

Undoable Drawing Pad

Goal	Create a minimal shape editor with select, move, delete, undo, and redo.
Learn	Model/view separation, commands, hit testing, reversible actions.
Constraints	Only rectangles; no persistence initially.
Done when	Undo restores both geometry and selection consistently; view can redraw entirely from model state.
Budget	8-14 hours
Stretch	Add save/load and command-history dirty tracking.

Usability Test and Revision

Goal	Give one prior UI to three people, observe five tasks, and revise based on evidence.
Learn	Task design, observation, usability defects, feedback loops.
Constraints	Do not instruct participants during tasks; record behavior, not inferred feelings.
Done when	Report includes task completion, observed errors, severity, changes, and before/after rationale.
Budget	5-8 hours
Stretch	Repeat one task after revision.

13. Performance and Reliability

Why it matters. Performance work starts with measurement, and reliability work starts with explicit failure modes. Fast incorrect software and robust software that cannot meet demand are both failures.

Target capability. You can benchmark repeatably, profile bottlenecks, define service-level goals, apply bounded retries, and design recovery from partial failure.

Profiler-Guided Optimization

Goal	Make a deliberately slow text/data processor materially faster without changing behavior.
Learn	Benchmarking, profiling, hotspots, algorithmic versus micro optimization.
Constraints	Record baseline before changes; optimize only measured hotspots.
Done when	Reports median and variability over repeated runs; tests prove output equivalence; improvement is at least 3x or explained.
Budget	5-8 hours
Stretch	Compare memory and CPU trade-offs.

Cache With Staleness Policy

Goal	Add a cache to an expensive data source with TTL, invalidation, and stale-on-error behavior.
Learn	Cache correctness, freshness, stampede risk, fallbacks.
Constraints	Make the clock injectable and policy explicit.
Done when	Tests cover hit, miss, expiry, invalidation, source failure, and concurrent refresh behavior.
Budget	6-10 hours
Stretch	Add single-flight refresh.

Retry and Circuit Breaker

Goal	Wrap a flaky dependency with bounded retries and a circuit breaker.
Learn	Transient versus permanent failure, backoff, jitter, state machine, recovery.
Constraints	No infinite retries; use a fake clock and scripted dependency.
Done when	Closed/open/half-open transitions are deterministic; retry budget is bounded; metrics expose behavior.
Budget	6-10 hours
Stretch	Add request deadlines.

Crash-Recovery Job Runner

Goal	Persist jobs so a process crash cannot silently lose accepted work.
Learn	Durability, leases, acknowledgements, replay, poison jobs.
Constraints	Simulate crashes at several write/execute boundaries.
Done when	Every accepted job is eventually completed or visibly dead-lettered; duplicate execution policy is documented.
Budget	8-14 hours
Stretch	Make handlers idempotent.

14. Requirements, Delivery, and Maintenance

Why it matters. Most professional work changes existing software under constraints. Good delivery turns ambiguous requests into testable outcomes, preserves compatibility, automates checks, and makes releases recoverable.

Target capability. You can write acceptance criteria, control scope, maintain compatibility, automate builds/tests, version interfaces, and plan migrations and rollback.

Ambiguous Requirement to Acceptance Tests

Goal	Take a vague feature request and turn it into examples, non-goals, risks, and executable acceptance tests.
Learn	Elicitation, ambiguity, boundary cases, acceptance criteria.
Constraints	Do not code until the brief is reviewed by another person or after a 24-hour self-review.
Done when	Every requirement maps to at least one acceptance test; unresolved assumptions are explicit.
Budget	3-5 hours
Stretch	Change one requirement and trace the affected tests and design.

Continuous Integration Pipeline

Goal	Create a repository where each push runs formatting/linting, tests, and artifact packaging.
Learn	Build automation, reproducibility, quality gates, artifacts.
Constraints	Use GitHub Actions, GitLab CI, or a local equivalent; dependencies are pinned.
Done when	A clean checkout can build with one command; a deliberate test failure blocks the pipeline.
Budget	4-7 hours
Stretch	Add a dependency vulnerability scan and coverage artifact.

Backward-Compatible File Format

Goal	Evolve a saved-document format through three versions without breaking older files.
Learn	Compatibility, deprecation, migration, semantic versioning.
Constraints	Preserve test fixtures for every released version.
Done when	Current code reads all historical fixtures; writers produce only the newest format; migration failures are safe.
Budget	5-8 hours
Stretch	Add a downgrade/export path.

Maintenance Change Simulation

Goal	Return to a project after four weeks and implement three change requests using only its repository documentation.
Learn	Maintainability, code archaeology, regression, documentation quality.
Constraints	Record time spent locating behavior versus changing behavior.
Done when	Changes pass regression tests; retrospective identifies documentation and design that helped or obstructed.
Budget	6-10 hours
Stretch	Ask another person to implement one change.

15. Professional Reasoning and Communication

Why it matters. Engineering judgment requires making uncertainty, trade-offs, evidence, and consequences visible. Communication is not decoration; it is part of system correctness and maintainability.

Target capability. You can write concise decision records, estimate with uncertainty, review for consequences rather than taste, explain systems at several levels, and identify ethical or operational risks.

Architecture Decision Record Set

Goal	Write five ADRs for a small system: context, decision, alternatives, consequences, and revisit trigger.
Learn	Decision quality, trade-offs, reversibility, documentation.
Constraints	Each ADR fits on one page and includes at least one rejected alternative.
Done when	A later reader can explain why the decision was reasonable at the time, even if they disagree.
Budget	3-5 hours
Stretch	Supersede one ADR after new evidence.

Estimate and Calibrate

Goal	Estimate ten small tasks using ranges, implement them, and compare predictions with actuals.
Learn	Uncertainty, decomposition, calibration, reference classes.
Constraints	Record estimates before work; do not revise them afterward.
Done when	Retrospective identifies systematic bias and produces one changed estimation rule.
Budget	Distributed across 2-3 weeks
Stretch	Repeat one month later and compare calibration.

Consequence-Based Code Review

Goal	Review a completed mini-project and classify findings by correctness, security, maintainability, performance, or preference.
Learn	Review scope, evidence, severity, actionable feedback.
Constraints	Every requested change must name a concrete consequence or violated project rule.
Done when	Review distinguishes must-fix from optional alternatives and includes at least one explicit “leave as is.”
Budget	2-4 hours
Stretch	Compare your review with a later external review.

Three-Level System Explanation

Goal	Explain one project to a user, a junior programmer, and an experienced engineer.
Learn	Audience, abstraction level, diagrams, terminology.
Constraints	One page per audience; no copy-pasting between versions.
Done when	Each explanation answers purpose, major flow, failure behavior, and how to verify it at the appropriate depth.
Budget	3-5 hours
Stretch	Record a five-minute spoken explanation.